Ishika
2301010242
Btech cse (sec-4)

CLASSTIME | Pg. No.
Date / /

Operating System

Capstone Assignment

Part A : Short Answer - Fundamental Concepts

1. Modern systems still need an OS because it provides:
   - Process management : scheduling, context switching, isolation.
   - Memory management : virtual memory, allocation, protection.
   - I/O management : device abstraction, interrupts, buffering.

   OS abstracts hardware complexity and ensures security, multitasking and resource sharing.

2. - Monolithic : all services in kernel; fast but less secure.
   - Layered : modules in layers; easier maintenance but slower.
   - Microkernel : only essentials in Kernel; highly reliable, modular, ideal for distributed systems.

   Choice for distributed web app : Microkernel, bcoz faults are isolated, services can restart independently and it supports modular scaling.

3. - Processes : need seperate PCB, full context switch, seperate memory → heavier

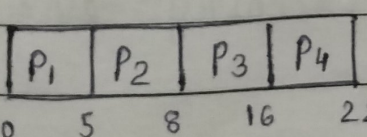
- Threads : share PCB components, cheaper context switch, share memory.
But thread management can cause synchronization issues.
Threads are generally more efficient, but not always safer.

4. Processes : 12 MB, 18 MB, 6 MB
Blocks : 20 MB, 10 MB, 15 MB
First - Fit :
- 12 MB → 20 MB (left : 8 MB)
- 18 MB → 15 MB ✗ too small → 10 MB ✗ → Not allocated
- 6 MB → 8 MB leftover block
Fragmentation : 8 MB unused, 10 MB unused.
Best - Fit :
- 12 MB → 15 MB (left : 3 MB)
- 18 MB → 20 MB (left : 2 MB)
- 6 MB → 10 MB (left : 4 MB)

Fragmentation : 3 MB + 2 MB + 4 MB (better utilization)

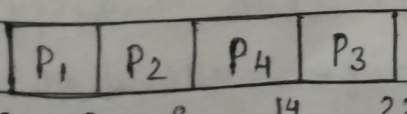
Part B : Simulation & Numerical Application

5.(a) 1. FCFS Scheduling

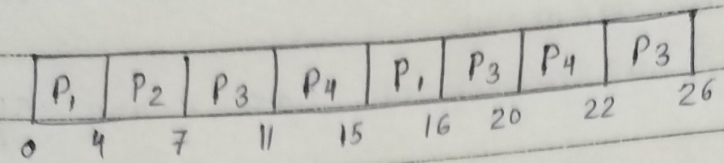


Gantt - chart

2. SJF (Non - Preemptive)



3. Round Robin ($q = 4$ ms)



(b) • Average WT = $\frac{(0+4+5+12)}{4}$ (SJF)

$= 5.25$ ms

Average TAT = $\frac{(5+7+11+20)}{4}$

$= 10.75$ ms

- FCFS :
  Average WT = $(0+4+6+13)/4 = 5.75$ ms
  Average TAT = $(5+7+14+19)/4 = 11.25$ ms
- Round Robin :
  Average WT = $(11+3+16+13)/4 = 10.75$ ms
  Average TAT = $(16+6+24+19)/4 = 16.25$ ms

(c) Best balance = Round Robin best for multiprogramming because
- Fair CPU sharing
- Good response time
- Prevents long jobs from blocking others.

6. (a) Banker's Algorithm :
Checks each request against a "safe state". Only grants if resources left can satisfy maximum future needs → avoids unsafe allocations.

(b) Detect + Recover :

- Maintain resource - allocation & request matrices.
- Run cycle - detection algorithm.
- If deadlock found → abort least-priority process or preempt resources.

7. Use semaphores:
"empty for free slots," full for filled slots and mutex for exclusive access. Product waits on empty, inserts item, signals full. Consumer waits on full, removes item, signals empty. Ensures synchronization & prevents race conditions.

8. Sequence: 2, 1, 4, 2, 3, 4, 3 ; Frames = 3
FIFO replaces the oldest loaded page; LRU replaces least recently used page. Both result in about 5 page faults here. LRU generally performs better in real usage due to locality of reference.

12. System calls shift execution from user mode to Kernel mode for safe hardware access. Functions like open(), write() invoke Kernel routines, allowing OS to manage file securely.

9. (a) Issues:

1. Consistency: same file accessed across locations.
2. Fault tolerance: node failure must not lose data.

(b) Architectures:

- Client-Server DFS (NFS-like): centralized directory service.
- Replica-based DFS (GFS/HDFS): chunk servers + replication → reliability, fast access.

10. How it works:

- All nodes pause → take checkpoint → resume.
- Ensures a consistent global state.

Pros: simple recovery, consistent.
Cons: high overhead, all processes stop.
Asynchronous: no global pause, but recovery is complex (domino effect).

11. (a) Scheduling Strategy:

- Use Priority Scheduling
  Security devices = highest priority
  Lighting = lowest priority

(b) IPC Methods:

- Message queues: structured data exchange.
- Signals / interrupts: for urgent device events.
- Shared memory + semaphores: for fast local communication.